B.TECH CSE — 5th SEMESTER LECTURE NOTES

Chapter 2
Intelligent Agents and Environments

Course: *Fundamentals of Machine Intelligence 1 (MTH 3647)*

Based on: *Artificial Intelligence and Machine Learning*

Pradeep Singh, Tapan K. Gandhi, Balasubramanian Raman



Contents

2 Intelligent Agents and Environments **3**

2.1 Introduction . . . 3

2.1.1 What is an Intelligent Agent? . . . 3

2.1.2 What is an Environment? . . . 3

2.1.3 What Does an Agent Actually Do? The Perceive–Reason–Act Cycle . . . 4

2.1.4 Percept and Action . . . 4

2.1.5 The Learning / Interaction Loop . . . 4

2.1.6 Policy . . . 5

2.1.7 Goal of an Intelligent Agent . . . 5

2.1.8 Maximizing Expected Utility . . . 6

2.1.9 Why Do We Need Mathematics? . . . 6

2.1.10 Real-World Examples of Intelligent Agents . . . 7

2.2 Intelligent Agents: Types and Structure . . . 7

2.2.1 Mathematical Representation of an Intelligent Agent . . . 7

2.2.2 Seeing the Environment: The Perception Equation . . . 8

2.3 Types of Intelligent Agents . . . 9

2.3.1 Reflex Agent . . . 9

2.3.2 Model-Based Agent . . . 9

2.3.3 Goal-Based Agent . . . 10

2.3.4 Utility-Based Agent . . . 10

2.3.5 Learning Agent . . . 11

2.3.6 Comparison of the Five Agent Types . . . 12

2.4 Applications of Intelligent Agents . . . 12

2.4.1 Autonomous Vehicles (Self-Driving Cars) . . . 12

2.4.2 Virtual Assistants . . . 14

2.5 Agents and Environments: Types and Structures . . . 15

2.5.1 Defining the Environment . . . 15

2.5.2 Discrete vs Continuous Environment . . . 18

2.5.3 Static vs Dynamic Environment . . . 18

2.5.4 Summary: Classifying an Environment . . . 19

2.5.5 Exploration vs Exploitation . . . 20

2.5.6 How Intelligent Agents Perceive, Act, and Optimize 20
2.6 Multi-Agent Systems: Cooperative and Competitive Agents 21
2.6.1 Cooperative Agents 22
2.6.2 Competitive Agents 22
2.6.3 Inter-Agent Communication (Communication Between Agents) . 23
2.7 Concept of Rationality and Good Behaviour 23
2.7.1 Rationality and Decision Making 23
2.7.2 Expected Utility 24
2.7.3 Choosing the Best Action 25
2.7.4 Making Decisions Over Time 26
2.7.5 Adding Utilities 26
2.7.6 Linear Programming 27
2.7.7 Dynamic Programming / Bellman Equation 27
2.7.8 Handling Uncertainty: Belief 28
2.7.9 Updating Belief 29
2.7.10 Acting Based on Belief 30
2.7.11 Measuring AI Performance 30
2.7.12 Classification Metrics 32
2.7.13 Efficiency 34
2.8 Robustness, Fairness, Safety, and Ethics in AI 35
2.8.1 Robustness 35
2.8.2 Fairness and Safety 35
2.8.3 Safety 38
2.8.4 Ethical Utility 38
2.9 Types of Intelligent Agents 38
2.9.1 Goal-Based Agent 38
2.9.2 Utility-Based Agent 38
2.9.3 Learning Agent 39
2.9.4 Multi-Agent Learning 40
2.10 Common Pitfalls 41
2.11 Practice Problems for Self-Study 42
2.12 Key Terms Glossary 43
2.13 Chapter Summary 45

Intelligent Agents and Environments

What this chapter covers: What an intelligent agent is, how it perceives, reasons, and acts; how percepts, actions, rewards, and policies relate to one another; the formal mathematical representation of an agent as (S, A, P, f, U); the five major agent types (reflex, model-based, goal-based, utility-based, learning); the mathematical structure of an environment (S, A, P, R) and its properties (discrete/continuous, static/dynamic, exploration/exploitation); how agents perceive, act, and maintain a belief state under uncertainty; multi-agent systems, cooperative and competitive agents, and inter-agent communication; and the concept of rationality and expected utility.

Why it matters: Every AI system — a self-driving car, a game-playing bot, a recommendation engine, or a virtual assistant — can be understood as an *agent* interacting with an *environment*. This chapter builds the formal vocabulary (percept, action, state, policy, utility, reward) used throughout the rest of the course, and is foundational for reinforcement learning, search, and planning.

2.1 Introduction

2.1.1 What is an Intelligent Agent?

Definition 1: Intelligent Agent is a system that can observe its surroundings, make decisions, and take actions to achieve a goal.

✎ **Example:** Consider a self-driving car.

- It **observes** the road using cameras and sensors.
- It **thinks/decides** whether to stop, turn, or accelerate.
- It **acts** by applying brakes, turning the steering wheel, etc.

2.1.2 What is an Environment?

Definition 2: Environment is everything around the agent that the agent can observe and interact with.

✎ **Example:** Self-driving car. **Agent:** Self-driving car. **Environment:** Road, vehicles, pedestrians, traffic lights, weather.

2.1.3 What Does an Agent Actually Do? The Perceive–Reason–Act Cycle

An intelligent agent performs three important activities in a repeating cycle:

1. **Perceive** – the agent observes the environment. *E.g. a self-driving car detects "There is a red traffic light."* This observation is called a **percept**.
2. **Reason / Decide** – the agent thinks: "The traffic light is red, so I should stop."
3. **Act** – the agent performs an action: apply brakes → car stops.

2.1.4 Percept and Action

Definition 3: Percept is the information an agent receives from its environment. *Percept = what the agent observes.*

Definition 4: Action is what the agent does after making a decision.

✎ **Example:** A self-driving camera senses an obstacle ahead — that information is a percept.

Percept: Red traffic light → Decision: Stop → Action: Apply brakes

2.1.5 The Learning / Interaction Loop

The basic cycle by which an agent interacts with its environment is:

$$\text{Environment} \rightarrow \text{Percept} \rightarrow \text{Agent} \rightarrow \text{Action} \rightarrow \text{Environment}$$

When the system uses rewards, the loop extends to:

$$\text{Environment} \rightarrow \text{Observation} \rightarrow \text{Agent} \rightarrow \text{Action} \rightarrow \text{Environment} \rightarrow \text{Reward}$$

This process continues again and again.

Definition 5: Reward is feedback given to an agent about how good or bad its action was. It tells the agent whether its action was good or bad.

✎ **Example:** An AI learning to play a game.

- Wins the game → +100 reward
- Loses the game → −100 reward
- Makes a good move → +10
- Makes a bad move → −10

The agent tries to learn actions that give it higher rewards over time.

2.1.6 Policy

Definition 6: Policy is a strategy that tells an intelligent agent which action to choose for a given situation. *Policy = strategy used by an agent to choose an action.*

Situation	Action
Traffic light is red	Stop
Traffic light is green	Move
Obstacle ahead	Turn
Destination reached	Stop

2.1.7 Goal of an Intelligent Agent

The agent does not simply take random actions — it has a **goal**, and it chooses actions that help it achieve that goal.

Agent	Goal
Self-driving car	Reach the destination safely.
Vacuum-cleaning robot	Clean the room.
Delivery robot	Deliver the package to the correct location.

2.1.8 Maximizing Expected Utility

Definition 7: Maximizing Expected Utility means choosing actions that are expected to produce the best overall outcome, where **Utility** = how desirable or useful an outcome is.

✎ **Example:** A self-driving car has two choices.

Option A: Reach destination quickly but take a dangerous road.
Option B: Take a little longer but use a safe road.
The agent should consider which choice gives the better overall result.

2.1.9 Why Do We Need Mathematics?

An intelligent agent must make decisions, but the real world is not always predictable. *E.g. a self-driving car sees a person crossing the road — it cannot always predict what the person will do next.* Since the real world is unpredictable, **mathematics helps the agent make better decisions.**

Several mathematical areas are useful:

- **Probability** – helps deal with uncertainty / unpredictable situations. *E.g. "There is an 80% probability that it will rain" — the agent can make a decision based on this uncertainty.*
- **Machine Learning** – helps the agent learn from data and past experience. *E.g. a robot learns from previous movements which path is better for reaching its destination.*
- **Optimization** – helps the agent find a better or best decision.

2.1.10 Real-World Examples of Intelligent Agents

- **Self-driving cars** – Observe: road, vehicles, pedestrians. Decide: stop, accelerate, turn. Act: steering, braking, acceleration.
- **Robot** – observes objects, understands its surroundings, decides what to do, and moves its arms or body.
- **Video game AI** – an enemy character observes the player's position, decides whether to attack or escape, moves toward the player, and changes its behaviour according to the situation.

Exam tip: The core cycle *Perceive* → *Reason/Decide* → *Act* and the extended loop *Environment* → *Percept* → *Agent* → *Action* → *Environment* → *Reward* are extremely common short-answer questions. Be ready to instantiate them with a self-driving car example.

2.2 Intelligent Agents: Types and Structure

Working as a Team & Distributed Decision-Making

Agents can **coordinate** their actions instead of duplicating effort. *E.g. 3 robots need to deliver 30 packages: Robot A → 10, Robot B → 10, Robot C → 10.* Dividing the work makes the system more efficient.

Definition 8: Distributed Decision-Making means each agent makes its own decision and works with other agents to achieve the goal: *each agent decides → agents work together → goal is achieved.*

Cooperative AI

Definition 9: Cooperative AI means agents work together and help each other to achieve a goal.

E.g. Three robots cooperate to move a heavy object that one robot alone cannot move: Robot A + Robot B + Robot C → move the object.

2.2.1 Mathematical Representation of an Intelligent Agent

An intelligent agent can be represented using **five components**:

$$\text{Agent} = (S, A, P, f, U)$$

- $S \rightarrow$ **States:** the current situation of the environment.

- $A \rightarrow$ **Actions:** all possible actions the agent can perform.
- $P \rightarrow$ **Transition Function:** rules that tell how an action changes the current state.
- $f \rightarrow$ **Policy:** tells the agent what action to take, $f : S \rightarrow A$.
- $U \rightarrow$ **Utility Function:** gives a score showing how good or bad a situation is, $U : S \rightarrow \mathbb{R}$ (real number). Higher utility = better situation; lower utility = worse situation.

Goal of the Intelligent Agent: The Optimal Policy

The agent tries to find the best policy f^* that gives the highest expected total utility over time:

$$f^* = \arg\max_f \ \mathbb{E}\Big[\sum_{t=0}^{T} \gamma^t U(s_t) \ \Big|\ f\Big]$$

- $f^* \rightarrow$ Best / optimal policy $\arg\max \rightarrow$ find the choice that gives the maximum value
- $\mathbb{E} \rightarrow$ Expected value $t \rightarrow$ Time step $T \rightarrow$ Final time step
- $\gamma \rightarrow$ Discount factor $U(s_t) \rightarrow$ Utility of state at time t

Definition 10: Discount Factor (γ) tells the agent how much importance it gives to future utility compared with current utility, where $0 \leq \gamma \leq 1$.

✎ **Example:** A student has two choices. Get Rs. 100 today, or Rs. 150 next month. The student asks: "Is waiting for the future reward worth it?" The discount factor γ helps the agent decide how much importance to give the future reward. If $\gamma \approx 0$, the agent cares more about immediate results (focuses more on now); if $\gamma \approx 1$, the agent cares more about future results.

2.2.2 Seeing the Environment: The Perception Equation

An agent may not be able to see everything in the environment.

$$p = g(s)$$

Here g represents the sensor/observation process that converts the actual state s into what the agent can perceive p.

- $s \rightarrow$ Actual state of the environment

- $g \rightarrow$ Perceptual function (how the agent observes the environment)
- $p \rightarrow$ Percept / observation (what the agent sees)

✎ **Example:** Suppose the actual environment is $s =$ Car + road + pedestrian + traffic light. The car's camera cannot see everything — it only detects $p =$ Road + traffic light. So $p = g(s)$.

2.3 Types of Intelligent Agents

There are five progressively more capable types of agents:

$$\text{Reflex} \rightarrow \text{Model-Based} \rightarrow \text{Goal-Based} \rightarrow \text{Utility-Based} \rightarrow \text{Learning}$$

2.3.1 Reflex Agent

Definition 11: Reflex Agent looks at the current percept and immediately chooses an action; it uses only the current situation, does not remember the past, and relies on simple predefined rules.

$$f(p) = a$$

$p \rightarrow$ percept / what the agent sees; $f \rightarrow$ rule that decides what to do; $a \rightarrow$ action.

✎ **Example:** Reflex Agent – self-driving car.

Suppose a self-driving car observes $p =$ Red traffic light.

The policy says $f(\text{Red Light}) = \text{Stop}$.

Therefore $a = \text{Stop}$.

In short: *What the agent sees → policy decides → agent takes action.*

2.3.2 Model-Based Agent

Definition 12: Model-Based Agent remembers past information and combines it with the current observation to decide what to do.

$$\hat{s}_{t+1} = h(\hat{s}_t, a_t, p_{t+1})$$

New Memory = Old Memory + Action + New Observation.

- $\hat{s}_t \rightarrow$ Old memory $a_t \rightarrow$ Action taken $p_{t+1} \rightarrow$ New observation
- $\hat{s}_{t+1} \rightarrow$ Updated memory / new memory $h \rightarrow$ Function that updates the memory

✎ **Example:** Model-Based Agent – self-driving car.

Remembers: there was an obstacle on the left. Sees now: another vehicle is ahead.

Decides: move to the right. Action: turns right.

Past information + current observation → decision → action.

2.3.3 Goal-Based Agent

Definition 13: Goal-Based Agent is an agent that takes actions to achieve a specific goal.

$$g(s) = \text{True}$$

When the agent reaches a state where its goal is satisfied, the goal condition becomes True — "I have reached my goal."

✎ **Example:** Goal-Based Agent – delivery robot.

Goal: deliver a package at Room 101.

The robot: Move → Turn → Move → Reach Room 101.

When it reaches Room 101: $g(s) = \text{True}$.

2.3.4 Utility-Based Agent

Definition 14: Utility-Based Agent chooses the best action that gives the best result.

$$f(p) = \arg\max_{a \in A} \mathbb{E}\big[U(s_{t+1}) \mid s_t, a\big]$$

- $f(p) \rightarrow$ Policy: the action chosen from the current percept
- $p \rightarrow$ Current percept / observation $\quad a \rightarrow$ One possible action $\quad A \rightarrow$ Set of all possible actions
- $\arg\max \rightarrow$ Choose the action that gives the highest value
- $\mathbb{E} \rightarrow$ Expected value $\quad U \rightarrow$ Utility function $\quad s_t \rightarrow$ Current state $\quad s_{t+1} \rightarrow$ Next state after taking the action

The equation means: *"What utility do I expect to get if I take action a in the current state?"* Then $\arg\max_{a \in A}$ means *"From all possible actions, choose the one with the highest expected utility."* So the whole equation says: the agent looks at the current

situation, considers possible actions, compares their expected results, and chooses the action with the highest utility.

✎ **Example:** Utility-Based Agent – choosing the safer road.

Goal-based agent asks: "Can I reach my destination?"

Utility-based agent asks: "Which option is the best way to reach my destination?"

A self-driving car has two possible routes:

Action a_1: first road → Utility = 70;

Action a_2: second road → Utility = 90.

The agent chooses $\arg\max(70, 90)$, therefore it chooses Action a_2, the safer road.

Utility-Based Agent = compare possible actions → find the best result → choose the best action.

2.3.5 Learning Agent

Definition 15: Learning Agent is an agent that learns from its experience and improves its decisions over time.

$$\pi^* = \arg\max_{\pi} \mathbb{E}\Big[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\Big]$$

- $\pi^* \rightarrow$ Best / optimal policy $\quad \pi \rightarrow$ Policy / rule / strategy
- $\arg\max \rightarrow$ Find the policy that gives the highest value $\quad \mathbb{E} \rightarrow$ Expected value / average
- $\Sigma \rightarrow$ Add all rewards $\quad t \rightarrow$ Time step $\quad T \rightarrow$ Final time step $\quad \gamma \rightarrow$ Discount factor
- $R(s_t + a_t) \rightarrow$ Reward received for taking action a_t in state s_t

✎ **Example:** Learning Agent – an AI playing a game.

At first, it makes many mistakes.

After playing many times, it learns which actions are better.

Eventually, it becomes better at playing the game.

Exam tip: Remember the progression **Reflex → Model-Based → Goal-Based → Utility-Based → Learning** together with each agent's characteristic equation – $f(p)=a$; $\hat{s}_{t+1}=h(\hat{s}_t, a_t, p_{t+1})$; $g(s)=\text{True}$; $f(p)=\arg\max_a \mathbb{E}[U(s_{t+1})|s_t, a]$; $\pi^*=\arg\max_\pi \mathbb{E}[\sum \gamma^t R(s_t+a_t)]$ – as these are frequently asked to be matched or derived.

2.3.6 Comparison of the Five Agent Types

Agent Type	Uses Past Memory?	Characteristic Equation	Decision Basis
Reflex	No	$f(p) = a$	Current percept only
Model-Based	Yes	$\hat{s}_{t+1} = h(\hat{s}_t, a_t, p_{t+1})$	Past memory + current observation
Goal-Based	Yes	$g(s) = \text{True}$	Whether the goal is satisfied
Utility-Based	Yes	$f(p) = \arg\max_a \mathbb{E}[U(s_{t+1}) \mid s_t, a]$	Highest expected utility
Learning	Yes, improves over time	$\pi^* = \arg\max_\pi \mathbb{E}[\sum_t \gamma^t R(s_t,a_t)]$	Experience-driven policy improvement

2.4 Applications of Intelligent Agents

There are **two important examples**:

- Autonomous Vehicles (Self-Driving Cars)
- Virtual Assistants

2.4.1 Autonomous Vehicles (Self-Driving Cars)

A **self-driving car** is an **intelligent agent** because it:

Percept → Decides → Acts

✎ **Example:** Traffic Light Response.

See traffic light → Decide whether to stop → Apply brakes

Perception Model (Seeing the World)

A self-driving car uses: **Cameras, Radar, GPS** and **Other sensors**. The **car** uses **sensors** to **observe the environment**.

✎ **Example:** Perception Pipeline.

Actual environment: Pedestrian + traffic light + other cars

Sensors detect: Pedestrian + traffic light

So: **Actual world → Sensors → Perception**

- **Tracking Moving Objects – Kalman Filter**

  A **car** needs to track **moving objects**.

  ✎ **Example:** Tracking a Moving Vehicle.

  The car in front of me is moving forward.

- **Object Recognition**

  The **car** needs to recognize things such as – **Pedestrian, Car, Bicycle, Traffic light,** and **Road Sign**.

  Neural Network can be used to **recognize these things**.

Decisions Making

Now the **car** has to decide: "What should I do?"

- **Safety Reward** The **car** should not get too **close** to another vehicle.

  ✎ **Example:** Safety Penalty.

  The **car is too close**, so it receives a **negative penalty**.

Control Strategies : Controlling the Car – MPC (Model Predictive Control)

MPC helps the car **look ahead**, choose the **best actions**, and **control the car safely**. It chooses **future actions** that **produce** the **lowest total cost**.

The **cost** could represent:

- Collision risk
- Energy consumption
- Sudden movements
- Poor driving

2.4.2 Virtual Assistants

Examples: Siri, Google Assistant, Alexa, and other AI assistants. A **virtual assistant** is also an **Intelligent agent**. It:

Receives your request → Understands it → Decides what to do → Performs the task

Natural Language Understanding (NLU)

A **virtual assistant** first needs to understand **what the user wants**.

✎ **Example:** Setting an Alarm. Suppose we say: "Set an alarm for 7 AM tomorrow."

The **assistant** needs to understand:

- **Intent (I)** → What the user wants → Set an alarm
- **Entity (E)** → Important details → 7 AM, tomorrow
- **Query (q)** → The user's complete request

- **Training the Assistant**

  Training means **giving the AI many examples** so that it learns how to **understand user requests correctly**.

  ✎ **Example:** Training Examples. Here we give it **many requests**:

  – "Set an alarm for 7 AM."

  – "Remind me at 8 AM tomorrow."

  – "Wake me up at 6 AM."

  The **assistant** learns the **patterns** in these examples.

Dialogue Management

The **assistant** must decide: "What should I say or do next?" It chooses the **best response/action** to provide the **best overall result** for the user.

✎ **Example:** Booking a Flight. User: "Book me a flight."

Assistant needs to **decide** what information to ask for:

- "What is your destination?"
- Then: "What date do you want to travel?"

The **assistant** chooses **actions** that **help complete the task**.

Task Execution and Learning

Suppose we tell the **assistant**: "Schedule a meeting with Rahul tomorrow at 10 AM."

The **assistant** may need to perform several actions: $a_1, a_2, \ldots, a_n$

✎ **Example:** Scheduling a Meeting.

- a_1 = Find Rahul's calendar
- a_2 = Check 10 AM availability
- a_3 = Create meeting
- a_4 = Send invitation

The **goal** condition is: $g(a_1, a_2, \ldots, a_n) = True$

The task has been **successfully completed**.

- **Learning from Mistakes**

  Suppose we tell the **assistant**:

  "No, I wanted 10 PM, not 10 AM."

  The **assistant** receives **feedback**. A **learning system** can use such **feedback** to **improve its future behavior**.

  Mistake → Feedback → Learning → Better future decisions

2.5 Agents and Environments: Types and Structures

2.5.1 Defining the Environment

Definition 16: Environment is everything around the agent that the agent can interact with.

✎ **Example:** Video game.

Agent → Game character;

Environment → Game world;

Actions → Move, jump, attack;

Reward → Points for good action.

✎ **Example:** Self-driving car.

Agent → Self-driving car;

Environment → Road, vehicles, traffic lights, pedestrians;

Actions → Brake, accelerate, turn.

Mathematical Representation of an Environment

An environment can be represented as:

$$\epsilon = (S, A, P, R)$$

- $S \rightarrow$ States: possible situations
- $A \rightarrow$ Actions: possible actions
- $P \rightarrow$ Transition Probability: chance of moving to another state
- $R \rightarrow$ Reward: score / feedback received

✎ **Example:** A game character is standing near a coin.

State S: character is near the coin;

Action A: move right;

Transition Probability P: 90% chance the character reaches the coin;

Reward R: +10 points for collecting the coin.

Transition Probability

The chance of moving an agent from the current state to the next state after taking an action:

$$P(s' \mid s, a)$$

"What is the chance of reaching the next state s', given the current state s and action a?" Here $s \to$ current state, $a \to$ action taken, $s' \to$ next state, $P \to$ probability / chance.

✎ **Example:** Transition probability of a robot moving rooms.

A robot is in Room A.

Current state $s =$ Room A;

action $a =$ Move right.

There are two possible results: 80% chance → Room B; 20% chance → Room C.

So $P(\text{Room B} \mid \text{Room A}, \text{Move right}) = 0.8$

the chance of reaching the next state s', given the current state s and action a, is 0.8.

Reward Function

The reward function is $R(s, a, s')$: it tells the agent how good or bad the result was.

✎ **Example:** Self-driving car.

Reaches destination safely → +100;

reaches destination slowly → +50;

gets into an accident → −100.

Goal of the Agent (Environment View)

The agent wants to get the highest total reward over time:

$$\pi^* = \arg\max_{\pi} \mathbb{E}\Big[\sum_{t=0}^{T} \gamma^t R(s_t, a_t, s_{t+1})\Big]$$

Here we find the best policy π^* that gives the highest total reward over time: $\pi^* \to$ best policy; $\pi \to$ possible policy; $\arg\max \to$ find the best policy; $\gamma \to$ discount factor; $R \to$ reward; $s_t \to$ current state; $a_t \to$ action; $s_{t+1} \to$ next state; $T \to$ final time /

step; $\mathbb{E} \rightarrow$ expected value.

2.5.2 Discrete vs Continuous Environment

Environments can also be classified based on the type of states and actions they have. There are two important types: **Discrete Environment** and **Continuous Environment**.

Discrete Environment

Definition 17: Discrete Environment is an environment where the states and actions are separate and countable.

Examples: Chess, Tic-Tac-Toe, board games.

✎ **Example:** Chess.

A Chessboard has 64 fixed squares,

It has fixed pieces, and a piece can make only specific legal moves.

So chess is a discrete environment.

Continuous Environment

Definition 18: Continuous Environment is an environment where states and actions can have many possible values within a range.

✎ **Example:** Driving a car.

A car's speed could be 10 km/h, 10.5 km/h, 10.55 km/h, 10.555 km/h, ...

there are many possible values, and similarly the steering angle can change by very small amounts.

So driving is a continuous environment.

For a car: State s = speed, position, steering angle;

Action a = acceleration, steering.

These values can change smoothly.

2.5.3 Static vs Dynamic Environment

Another important classification is **Static Environment** vs **Dynamic Environment**.

Static Environment

Definition 19: Static Environment does not change by itself; it changes only when the agent performs an action.

✎ **Example:** A puzzle. If you don't touch the puzzle, nothing changes.
✎ **Example:** Chess. If a particular legal move is made in a particular position, the rules of chess do not change tomorrow.

Dynamic Environment

Definition 20: Dynamic Environment changes over time, even if the agent does nothing.

✎ **Example:** A self-driving car is waiting at a traffic signal and does nothing.

But other cars move, pedestrians move, traffic lights change, and weather may change,

therefore, the environment changes on its own.

✎ **Example:** Morning: heavy traffic → 30% chance of reaching quickly.

By afternoon this probability can change because the environment itself has changed.

2.5.4 Summary: Classifying an Environment

Classification	Test	Example
Discrete	States/actions are separate & countable: $S = \{s_1, \ldots, s_n\}$	Chess, Tic-Tac-Toe
Continuous	States/actions take real values in a range: $S \subseteq \mathbb{R}^n$	Driving a car, drone flight
Static	$P(s' \mid s, a) = P_0(s' \mid s, a)\ \forall t$ (unchanged by itself)	Solving a puzzle
Dynamic	$P_t(s' \mid s, a) \neq P_{t+1}(s' \mid s, a)$ (changes over time on its own)	Waiting at a signal in live traffic

2.5.5 Exploration vs Exploitation

Dynamic environments require the agent to adapt. There are two important ideas:

- **Exploitation** – use what the agent already knows works. *E.g. "This road usually gets me to college quickly, so I'll use it."*
- **Exploration** – try something new to find a better option. *E.g. "I will try another road today to see if it is faster."*

> **Exam tip:** *Exploitation* = using known-good actions; *Exploration* = trying new actions to potentially find something better. This trade-off is central to reinforcement learning.

2.5.6 How Intelligent Agents Perceive, Act, and Optimize

This section explains the complete working cycle of an intelligent agent:

$$\text{Perceive (see)} \rightarrow \text{Understand} \rightarrow \text{Act} \rightarrow \text{Learn/Improve}$$

How Agents "See" the World: Perception

An agent uses sensors to observe the environment, but sensors are not always perfect — a camera may produce a blurry image, or a sensor may give an incorrect measurement.

$$p_t = g(s_t) + \epsilon_t$$

$s_t \rightarrow$ actual state of the environment; $g \rightarrow$ perceptual function / observation process; $p_t \rightarrow$ percept/observation received by the agent; $\epsilon_t \rightarrow$ sensor noise / error; $t \rightarrow$ current time. The agent observes the environment, but its observation may contain some error.

> ✎ **Example:** Actual object is 20 metres away,
>
> but the sensor says the car is 20.5 metres away.
>
> The difference is sensor noise ϵ_t.
>
> So: *Actual state + sensor error → percept.*

Belief State

Because the agent's sensors are not perfect, the agent may not know the exact state, so it maintains a **belief state**.

Definition 21: Belief State is the agent's best guess about the current state of the environment.

✎ **Example:** A robot cannot see behind a wall.

The robot thinks: "There is probably an object behind the wall."

That probability-based guess is its belief.

How Agents "Do" Things: Action

After observing the environment, the agent takes an action using **actuators** (action devices like motor, brake, robotic arm). *E.g. Robot → Motor; Car → Steering/brakes; Robotic arm → mechanical joints.*

When the agent takes an action, the environment changes.

✎ **Example:** Current state: car is moving at 40 km/h;

action: press the brake;

next state: car slows down.

But the result may not be perfectly predictable because of road conditions,

so there may be some error η_t.

2.6 Multi-Agent Systems: Cooperative and Competitive Agents

Definition 22: Multi-Agent System (MAS) is an environment where two or more AI agents interact with each other.

✎ **Example:** Think of it like a classroom.

Agent 1 → Student A;

Agent 2 → Student B;

Environment → Classroom.

Agent 1 and Agent 2 can communicate, cooperate, or compete.

- **Definition 23: Shared Environment:** In a multi-agent system, multiple agents interact with a common environment.

2.6.1 Cooperative Agents

Definition 24: Cooperative Agents are types of agents that work like a team.

✎ **Example:** Two delivery robots working together.

Robot 1 delivers packages in Area A;

Robot 2 delivers packages in Area B.

Both want to complete all deliveries quickly;

they have a common goal, so they try to maximize a shared reward.

✎ **Example:** Suppose two robots deliver 10 packages.

Robot 1 delivers 5,

Robot 2 delivers 5,

total reward = 10.

Both robots are interested in:

"How can we work together to increase the total reward?"

2.6.2 Competitive Agents

Definition 25: Competitive Agents are agents that do not have the same goal. — They are like two players playing chess: Agent 1 wants to win, Agent 2 wants to win; if Agent 1 wins, Agent 2 loses.

Each agent therefore tries to maximize its own payoff.

✎ **Example:** Chess AI. Two chess-playing AI agents:

Agent 1 wants checkmate;

Agent 2 wants checkmate.

Both predict the opponent's possible moves, and each chooses its best strategy.

So:

Cooperation → maximize shared reward; Competition → maximize individual payoff.

2.6.3 Inter-Agent Communication (Communication Between Agents)

Definition 26: Inter-Agent Communication means agents communicate with each other by exchanging information or messages.

E.g. Agent A: "Obstacle detected." → Agent B: "I will take another route."

Agents often need to **communicate information**.

✎ **Example:** Two Autonomous Cars at an Intersection. Two autonomous cars are approaching an **intersection**.

- **Car A** can send the **information** to Car B: "I am turning left."
- **Car B** receives this **information** and changes its **behavior**.

Communication therefore has **two basic stages**:

1. Send a message
2. Receive and update information

✎ **Example:** Belief Update – Two Self-Driving Cars. **Before** receiving the message:

- **Car B** thinks: "Car A may go straight."

Message received: Car A → "I am turning left."

After receiving the message:

- Car B updates its belief: "Car A is turning left."

2.7 Concept of Rationality and Good Behaviour

2.7.1 Rationality and Decision Making

Definition 27: Rationality in AI means choosing the action that is expected to give the best outcome.

✎ **Example:** A self-driving car has two choices:

Stop

Continue.

If **the traffic light is red**, a rational agent chooses *Stop* → the safer/better outcome.

Definition 28: Utility Function $U : S \to \mathbb{R}$ gives a numerical value to a state, expressing how desirable that state is.

Situation	Utility
Reaching Destination Safely	+100
Reaching Destination Late	+20
Accident	−100

2.7.2 Expected Utility

The real world is unpredictable / uncertain.

Definition 29: Expected Utility is calculated as the probability of an outcome multiplied by the utility of that outcome, added together for all possible outcomes.

$$\mathbb{E}[U \mid s, a] = \sum_{s' \in S} P(s' \mid s, a)\, U(s')$$

$s \to$ current state;

$a \to$ action being considered;

$s' \to$ possible next state;

$P(s' \mid s, a)\, U(s') \to$ probability of reaching that state multiplied by the utility of the state;

$\Sigma \to$ add the values for all possible outcomes.

The AI considers every possible outcome, multiplies its probability by its utility, and adds them all together:

Expected Utility $= \sum$(Probability $\times$ Utility).

Solved Problem 1: Expected utility – self-driving car choosing to continue.

Suppose a self-driving car chooses to continue.

There are two possible outcomes:

80% chance → reaches destination → utility = 100;

20% chance → accident → utility = −100.

Solution:

$$EU = (0.8)(100) + (0.2)(-100) = 80 - 20 = 60$$

The AI calculates the expected utility,

then adds it up for all possible outcomes, to get $EU = 60$.

Exam tip: The formula $\mathbb{E}[U \mid s, a] = \sum_{s'} P(s' \mid s, a)\, U(s')$ (Expected Utility = $\sum$(Probability × Utility)) is one of the most frequently tested equations in this chapter. Practice plugging in 2–3 outcome scenarios like the self-driving car example above.

2.7.3 Choosing the Best Action

Once the AI calculates **expected utility** for different actions, it chooses the action with the **highest expected utility**.

$$a^* = \arg\max_{a \in A} \mathbb{E}\big[\, U \mid s, a \,\big]$$

- a^* = Best / optimal action.
- A = Set of all possible actions.
- s' = Possible next state.
- $\mathbb{E}(U \mid s, a)$ = **Expected Utility** of an action.
- $\arg\max$ = Find the action that gives the **maximum value**.

Solved Problem 2: Choosing the best action by expected utility.

Action	Expected Utility
Stop	70
Continue	60
Turn	40

The AI chooses $a^* =$ Stop, because 70 is the highest expected utility.

2.7.4 Making Decisions Over Time

- Sometimes an **AI** doesn't make just **one decision**.
- It makes a **sequence of decisions**.
- The AI needs to consider both:
  - **Immediate reward**
  - **Future reward**

2.7.5 Adding Utilities

When an **agent** passes through **multiple states**, its **total utility** is the **sum of the utilities** of those **states**.

$$U(s_1, s_2, \ldots, s_n) = \sum_{i=1}^{n} U(s_i)$$

- $s_1, s_2, \ldots, s_n =$ Different states visited by the agent.
- $U(s_i) =$ Utility of each state.
- $\sum \rightarrow$ Add all the utilities.

Solved Problem 3: Robot Journey. Suppose a **robot** moves through **three states**:

State	Utility
s_1	20
s_2	30
s_3	50

Therefore:

$$U(s_1, s_2, s_3) = 20 + 30 + 50$$

$$U(s_1, s_2, s_3) = 100$$

2.7.6 Linear Programming

Linear Programming finds the **best solution** while **obeying given limitations or constraints**.

✎ **Example:** Linear Programming – Event Budgeting.

Suppose we have $Rs.1{,}000$ and want to **buy food** for an **event**.

Here we want to **maximize the number of people served**,

but we cannot **spend more than *Rs.*1,000**.

So:

- **Goal:** Serve maximum people.
- **Constraints / Limitations:** $Budget \leq 1000$.

That is the **basic idea** behind **Linear Programming**.

2.7.7 Dynamic Programming / Bellman Equation

Dynamic Programming helps an AI make a **sequence of decisions** by considering both:

- **Immediate reward**
- **Future reward**

■ **Bellman Equation**

The Bellman Equation helps an AI choose the **best action** by considering both the **current reward** and the **future reward**.

$$V(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} P(s' \mid s,a)\, V(s') \right]$$

- $V(s)$ = Value of the current state
- $R(s,a)$ = Immediate reward
- a = Action
- γ = Discount factor
- $P(s' \mid s,a)$ = **Probability** of reaching s' after taking **action** a in **state** s
- $V(s')$ = Future value
- max = Choose the best action

✎ **Example:** Bellman Equation – Path Choice. We have two choices:

Path A:

- Immediate reward = 50
- Future reward = 20

Path B:

- Immediate reward = 20
- Future reward = 100

If the AI only looks at the **immediate reward**: $50 > 20$, it would choose **Path A**.

But **Dynamic Programming** says: if **future rewards** are **sufficiently important**, then AI may choose **Path B**.

2.7.8 Handling Uncertainty: Belief

Sometimes an AI doesn't know the **exact state**.

Solved Problem 4: Noisy Sensor – Belief State.

A robot has a noisy sensor.

Robot doesn't know exactly: "Am I in **Room A** or **Room B**?"

Instead, it has a **belief**:

- Room A $\rightarrow b(A) = 0.7$
- Room B $\rightarrow b(B) = 0.3$

70% belief that it is in **A** and **30% belief** that it is in **B**.

2.7.9 Updating Belief

When the AI receives **new information**, it updates its **belief**. The equation will be:

$$b(s \mid p) = \frac{P(p \mid s)\, b_{prior}(s)}{P(p)}$$

$$b(s) \propto P(p \mid s)\, b_{prior}(s)$$

- $b(s)$ or $b(s \mid p)$ = **Updated belief** about state s
- $P(p \mid s)$ = **Probability** of receiving observation p if the agent is in state s
- $b_{prior}(s)$ = **Old belief** before receiving the observation
- p = New observation / percept

Old belief + New observation → Updated belief

This is based on **Bayesian reasoning**. The AI uses **new information** to update what it already **believes**.

Solved Problem 5: Posterior Belief Using Bayes' Rule. An agent has **three possible states** s_1, s_2, and s_3. The **prior belief** is **uniformly distributed** across the **three states**. After observing **percept** p, the likelihoods are:

$$P(p \mid s_1) = 0.7 \qquad P(p \mid s_2) = 0.2 \qquad P(p \mid s_3) = 0.1$$

Calculate the **posterior belief distribution** $b(s \mid p)$ or $b(s)$ using **Bayes' rule**.

Solution:

Step 1: Since the **prior** is uniformly distributed across 3 states:

$$b_{prior}(s_1) = b_{prior}(s_2) = b_{prior}(s_3) = \frac{1}{3} = 0.333$$

Step 2: The likelihoods of observing percept p are:

$$P(p \mid s_1) = 0.7 \qquad P(p \mid s_2) = 0.2 \qquad P(p \mid s_3) = 0.1$$

Step 3: Calculate the **probability of observation** p:

$$P(p) = P(p \mid s_1)\, b(s_1) + P(p \mid s_2)\, b(s_2) + P(p \mid s_3)\, b(s_3)$$

$$P(p) = (0.7 \times 0.333) + (0.2 \times 0.333) + (0.1 \times 0.333)$$

$$P(p) = 0.2331 + 0.0666 + 0.0333$$

$$P(p) = 0.333$$

Step 4: Calculate the **updated beliefs**:

$$b(s \mid p) = \frac{P(p \mid s)\, b_{prior}(s)}{P(p)}$$

For s_1: $b(s_1 \mid p) = \dfrac{0.7 \times 0.333}{0.333} = 0.7$

For s_2: $b(s_2 \mid p) = \dfrac{0.2 \times 0.333}{0.333} = 0.2$

For s_3: $b(s_3 \mid p) = \dfrac{0.1 \times 0.333}{0.333} = 0.1$

2.7.10 Acting Based on Belief

If the AI doesn't know about the **current state**, it uses its **beliefs** to decide **what action to take**.

✎ **Example:** Acting Under Uncertainty – Closed Door.

A robot sees a **closed door** but doesn't know **what is behind it**.

It believes:

- 70% → **Room is safe**
- 30% → **Room is dangerous**

The **robot** chooses: **Open the door** because it believes there is a **higher chance** that the room is safe.

2.7.11 Measuring AI Performance

After building an AI, we need to ask: "How well is my AI performing?" Different tasks require **different metrics**.

Reward Metrics

The **total reward** is the **overall reward collected** by the AI over time.

$$R_{total} = \sum_{t=0}^{T} \gamma^t \, R(s_t, a_t)$$

- R_{total} = Total reward
- t = Current time step
- T = Final time step
- $R(s_t, a_t)$ = **Reward received** at time t
- γ = Discount factor
- γ^t = Gives **less importance to rewards** received further in the **future**.
- $\sum \rightarrow$ Add all the rewards
- s_t = **Current state** at time t
- a_t = **Action** taken at time t

Solved Problem 6: Total Reward Calculation. Suppose a robot gets:

- Step 1 $\rightarrow$ reward = 10
- Step 2 $\rightarrow$ reward = 20
- Step 3 $\rightarrow$ reward = 30

Total reward $R_{total} = 10 + 20 + 30 = 60$.

Solved Problem 7: Discounted Total Reward Calculation. Consider an **AI agent** that takes **three steps** and receives the following rewards:

- **Step 0 (Now):** Immediate reward = 10
- **Step 1 (Next):** Immediate reward = 20
- **Step 2 (Later):** Immediate reward = 50

Suppose the developer sets the **discount factor** to: $\gamma = 0.5$.

Solution:

The expected cumulative reward:

$$R_{total} = \sum_{t=0}^{T} \gamma^t \, R(s_t, a_t) = (0.5^0 \times 10) + (0.5^1 \times 20) + (0.5^2 \times 50)$$

$$= (1 \times 10) + (0.5 \times 20) + (0.25 \times 50) = 32.5$$

Average Metrics

Average Reward tells us the **average amount of reward** the AI receives at **each time step**. The equation will be:

$$R' = \frac{1}{T} \sum_{t=0}^{T} R(s_t, a_t)$$

- R' = Average reward
- t = Current time step
- T = Total time step
- $R(s_t, a_t)$ = **Reward received** at time t
- $\sum \rightarrow$ Add all the rewards
- s_t = **Current state** at time t
- a_t = **Action** taken at time t

Solved Problem 8: Average Reward Calculation. Rewards: $10, 20, 30$.

Here:

- $T = 3 \rightarrow 3$ time steps
- $R(s_1, a_1) = 10$
- $R(s_2, a_2) = 20$
- $R(s_3, a_3) = 30$

Average reward $R' = \dfrac{10 + 20 + 30}{3} = 20$.

2.7.12 Classification Metrics

Classification metrics tell us **how well a classification model** is making **predictions**.

Solved Problem 9: Spam Classification. AI is predicting whether an **email** is **Spam** or **Not Spam**.

Accuracy

Accuracy means **out of all predictions, how many are correct**?

$$Accuracy = \frac{Correct\ Prediction}{Total\ Prediction}$$

Solved Problem 10: Accuracy Calculation. AI makes **100 predictions** and **90 are correct**.

$$Accuracy = \frac{90}{100} = 90\%$$

Precision

Out of everything the AI predicted as **positive**, how many were actually **positive**? When AI says **Positive**, how often is it correct?

$$Precision = \frac{TP}{TP + FP}$$

Solved Problem 11: Precision Calculation. AI says **10 emails are spam**.

- 8 are actually spam → **TP = 8**
- 2 are not spam → **FP = 2**

$$Precision = \frac{8}{8 + 2} = 80\%$$

Recall

Out of all the **actual positive cases**, how many did the AI **correctly find**? Out of all **actual positives**, how many did AI find?

$$Recall = \frac{TP}{TP + FN}$$

Solved Problem 12: Recall Calculation. There are actually **10 spam emails**. AI correctly detects: 8.

- TP = 8
- FN = 2

$$Recall = \frac{8}{8+2} = 80\%$$

F1 Score

F1 Score gives a **balanced measure** of Precision and Recall.

$$F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Solved Problem 13: F1 Score Calculation. If:

- Precision = 80%
- Recall = 80%

$$F1\ Score = 2 * \frac{80 * 80}{80 + 80} = 80\%$$

2.7.13 Efficiency

AI should not only be **accurate**. It should also **complete tasks quickly** and with **reasonable energy consumption**.

Time Efficiency

Time efficiency tells us the **average time** taken by the AI to complete a task.

$$T' = \frac{1}{N} \sum_{i=1}^{N} T_i$$

- T' = Average Time
- N = Number of tasks
- T_i = Time taken for task i
- $\sum$ = Add all task times

Energy Efficiency

Energy efficiency tells us **how much energy the AI uses** to achieve a **certain amount of reward**.

$$Energy\ Efficiency = \frac{Total\ Energy\ Spent}{Total\ Reward\ Achieved}$$

Solved Problem 14: Energy Efficiency Calculation. Suppose a robot:

- Uses **100 units** of energy
- Achieves **50 units** of reward

Then:

$$Energy\ Efficiency = \frac{100}{50} = 2$$

So the **robot uses 2 energy units per reward unit**.

2.8 Robustness, Fairness, Safety, and Ethics in AI

2.8.1 Robustness

Robustness means the AI can handle **changes or problems** without its **performance dropping** significantly.

Robust AI = Works well even when conditions change

$$Var(R_{total}) = \mathbb{E}[R_{total}^2] - (\mathbb{E}[R_{total}])^2$$

- R_{total} = Total reward
- $\mathbb{E}[R_{total}]$ = Average / Expected total reward
- $\mathbb{E}[R_{total}^2]$ = Expected squared total reward
- Var = Variance

Solved Problem 15: Variance and Consistency. Suppose an AI performs 3 times.

- **AI A:** $80, 82, 81$ – the scores are very close → **Low variance**.
- **AI B:** $40, 90, 60$ – the scores are very different → **High variance**.

Low Variance → More Consistent AI

2.8.2 Fairness and Safety

An AI should not only try to **maximize its reward**. It should also **make decisions** that are:

- **Fair**
- **Safe**
- **Ethical**

Fair Utility

The AI can include a **penalty** for **unfair behavior**.

$$Fair\ Utility(s,a) = f(s,a) - \lambda\, Fairness\ Penalty(s,a)$$

- $f(s,a)$ = Utility or score of the action
- $Fairness\ Penalty(s,a)$ = Penalty for unfair behavior
- λ (Lambda) = Controls how strongly fairness is considered

If an AI makes an **unfair decision**, its **utility** is reduced by a **fairness penalty**.

Solved Problem 16: Fair Utility – Job Selection. Suppose an AI is selecting **candidates for a job**. If an **unfair decision** gives: **Utility** = 100. But suppose the decision is found to have a **fairness penalty** = 20.

Let $\lambda = 1$:

$$Fair\ Utility = f(s,a) - \lambda \times Fairness\ Penalty = 100 - (1 \times 20)$$

$$Fair\ Utility = 80$$

So the **AI's score** is **reduced** because the **decision was unfair**.

Fair Utility = Normal Utility – Fairness Penalty

We can take **different values of** λ depending on how **strongly** we want fairness to importance of the decision.

Demographic Utility

Demographic parity is a **fairness condition** that aims for **similar positive-outcome rates** across **different groups**.

$$P(\hat{y} = 1 \mid group_1) = P(\hat{y} = 1 \mid group_2)$$

- $\hat{y} = 1$ = AI gives positive outcome.

- $group_1, group_2$ = Different groups
- P = Probability

Solved Problem 17: Demographic Parity – Interview Selection. Suppose an AI is selecting candidates for an interview.

- Group 1 → 60% selected
- Group 2 → 60% selected

Then: $P(\hat{y} = 1 \mid group_1) = P(\hat{y} = 1 \mid group_2) = 0.60$.

So the **selection rates** of both groups are **equal**, so it is a **Demographic Utility**.

Equal Opportunity

Equal Opportunity means that **qualified people** from **different groups** should have the **same chance** of being **selected**.

$$P(\hat{y} = 1 \mid y = 1, group_1) = P(\hat{y} = 1 \mid y = 1, group_2)$$

- $\hat{y} = 1$ = AI gives positive outcome.
- $y = 1$ = Person actually qualifies
- $group_1, group_2$ = Different groups
- P = Probability

Solved Problem 18: Equal Opportunity – Interview Selection. AI selecting candidates for an interview. Suppose we have qualified candidates only:

- Group 1: 100 qualified → 80 selected
- Group 2: 100 qualified → 80 selected

Then:

$$P(\hat{y} = 1 \mid group_1) = 0.80$$

$$P(\hat{y} = 1 \mid group_2) = 0.80$$

Therefore:

$$P(\hat{y} = 1 \mid group_1) = P(\hat{y} = 1 \mid group_2) = 0.80$$

Equal Opportunity means that **qualified people** from **different groups** should have the **same chance** of being **selected**.

2.8.3 Safety

An AI should **avoid dangerous actions**. Otherwise it will be considered as **risk**.

2.8.4 Ethical Utility

Ethical Utility combines **what is good for the AI's performance** with **what is good for people**.

2.9 Types of Intelligent Agents

2.9.1 Goal-Based Agent

A **goal-based agent** chooses **actions** that help it reach a **specific goal**. The agent asks: "What action will help me reach my goal?"

Solved Problem 19: GPS Navigation System. **Goal:** Reach **Bhubaneswar** → **Puri**.

The GPS considers **different routes** and chooses a **route** that reaches the destination.

Starting Point → Choose Route → Choose Next Action → Destination

Goal-Based Agent = Choose actions to reach a goal

2.9.2 Utility-Based Agent

A **utility-based agent** chooses the **action** that gives the **best overall outcome or highest utility**.

Solved Problem 20: GPS Route Selection. A GPS needs to find a **route** to a **destination**. It has three routes:

Route	Time	Cost	Utility
A	30 min	*Rs.*100	70
B	40 min	*Rs.*50	80
C	25 min	*Rs.*200	60

Even though **Route C** is the **fastest**, **Route B** has the **highest utility (80)**. So the AI chooses: **Route B**.

Because the **agent** considers **multiple factors**, such as: **Time, Cost, Traffic, Distance, Safety**.

Utility-Based Agent = Choose the action with the highest utility

2.9.3 Learning Agent

A **learning agent** improves its **behavior** by **learning from experience and feedback**. Instead of always following **fixed rules**, it can **learn** from its **mistakes and successes**.

Solved Problem 21: Self-Driving Car. The car receives:

- Reward → reaches destination safely
- Penalty → collision
- Penalty → unsafe driving
- Reward → efficient driving

Over time, it learns: "Which actions lead to better results?"

Environment → Action → Reward/Penalty → Learning → Better Action

Reinforcement Learning Agent

Reinforcement Learning (RL) is a type of **Machine Learning** in which an **agent** learns by **interacting with an environment** and receiving **rewards** or **penalties**.

Try an action → get a reward/penalty → learn from the result → improve the next action.

Solved Problem 22: Teaching a Dog to Sit. Imagine teaching a dog to sit.

- Dog sits → **Reward**
- Dog does not sit → No reward

After **repeated attempts**, the dog learns that sitting leads to a **reward**.

Expected Cumulative Reward was defined:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, R(s_{t+k}, a_{t+k})$$

- G_t = Cumulative Reward
- t = Current time
- k = Number of steps into the future
- γ = Discount factor
- R = Reward function
- s_{t+k} = State at time $t+k$
- a_{t+k} = Action at time $t+k$
- $\sum$ = Add all the rewards

The **agent** tries to **maximize the total reward** it expects to **receive now** and in the **future**.

Solved Problem 23: Cumulative Discounted Reward – Game Player. Suppose a **player** is playing a **game** and gets these rewards:

Step	Reward (R)
Current step ($k=0$)	10
Next step ($k=1$)	5
2 steps later ($k=2$)	20

Assume: **Discount factor** $\gamma = 0.9$.

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, R(s_{t+k}, a_{t+k})$$

$$G_t = (0.9^0 \times 10) + (0.9^1 \times 5) + (0.9^2 \times 20) = 10 + 4.5 + 16.2 = 30.7$$

So the **cumulative discounted reward** $G_t = 30.7$.

2.9.4 Multi-Agent Learning

Multi-agent learning occurs when **multiple AI agents** learn and change their behavior by **interacting with each other** in the **same environment**.

Solved Problem 24: Two AI Players in a Game. Imagine **two AI players** playing a game.

- **Agent 1** observes **Agent 2's** strategy.
- **Agent 2** observes **Agent 1's** strategy.

Both agents **learn** and **change their strategies** based on **each other's actions**.

Types of Multi-Agent Learning

There are **two** types of **Multi Agent Learning**.

Cooperative Learning **Agents** work together to **achieve a common goal**.

Solved Problem 25: Cooperative Robots. **Two robots** working together to move a heavy box.

Competitive Learning **Agents** compete and try to **maximize their own rewards**.

Solved Problem 26: Competitive Chess Agents. **Two AI agents** playing chess.

2.10 Common Pitfalls

- Confusing a **percept** (what the agent observes, p) with an **action** (what the agent does, a) – they are opposite ends of the perceive–act cycle.
- Thinking a **static** environment never changes – it only means the environment does not change *by itself*; it can still change when the agent acts.
- Forgetting that a **dynamic** environment can change even when the agent does nothing (e.g. traffic, weather).
- Mixing up **goal-based** agents ("Can I reach the goal?", a True/False condition $g(s)$) with **utility-based** agents ("Which option is best?", comparing $\arg\max$ over expected utilities).
- Forgetting the range of the discount factor: $0 \leq \gamma \leq 1$. A γ close to 0 favours immediate reward; a γ close to 1 favours long-term/future reward.
- Confusing **exploitation** (using a known-good action) with **exploration** (trying something new to potentially find a better action).
- Forgetting that **cooperative** agents maximize a *shared* reward while **competitive** agents maximize their *own individual* payoff.

2.11 Practice Problems for Self-Study

Solved Problem 27: Practice – classify the agent type. A thermostat immediately switches the heater ON whenever the room temperature reading falls below 18°C, using only the current sensor reading and a fixed rule. Which type of agent is this, and why?

Solution: The thermostat maps its current percept p (the temperature reading) directly to an action a (ON/OFF) using a fixed rule, with no memory of past readings and no comparison of expected outcomes. This matches the equation $f(p) = a$. **Hence it is a Reflex Agent.**

Solved Problem 28: Practice – compute expected utility. A delivery robot can choose between two routes. Route X: 90% chance of on-time delivery (utility = +50), 10% chance of a delay (utility = −20). Route Y: 70% chance of on-time delivery (utility = +50), 30% chance of a delay (utility = −20). Using $\mathbb{E}[U \mid s, a] = \sum_{s'} P(s' \mid s, a)U(s')$, which route should a rational agent choose?

Solution:

$$EU_X = (0.9)(50) + (0.1)(-20) = 45 - 2 = 43$$

$$EU_Y = (0.7)(50) + (0.3)(-20) = 35 - 6 = 29$$

Since $EU_X = 43 > EU_Y = 29$, a rational agent chooses the action that maximizes expected utility: $a^* = \arg\max_a \mathbb{E}[U \mid s, a]$. **Hence Route X is the better choice.**

Solved Problem 29: Practice – discrete or continuous? State whether each of the following environments is discrete or continuous, with a one-line justification: (a) a Sudoku puzzle, (b) a drone's flight path through open air, (c) an ATM PIN-entry keypad, (d) the temperature control of an oven over time.

Solution: Test each against $S = \{s_1, \ldots, s_n\}$ (discrete, countable) vs. $S \subseteq \mathbb{R}^n$ (continuous, real-valued):

(a) **Discrete** – each cell holds one of 9 fixed digits; the set of board configurations is finite and countable.

(b) **Continuous** – position, velocity, and orientation can take any real value within a range.

(c) **Discrete** – only 10 fixed digit buttons (0–9) can be pressed; the action set is finite.

(d) **Continuous** – temperature can vary smoothly over a real-valued range (e.g. 150.0°C, 150.1°C, ...).

Solved Problem 30: Practice – cooperative or competitive? Two chess-playing programs are matched against each other in a tournament. Two warehouse robots are jointly rewarded based on how many total packages the pair delivers per hour. Classify each scenario as a cooperative or competitive multi-agent system, and give the relevant equation.

Solution: The chess programs each want to win and the other to lose – their goals directly conflict, so each maximizes its own payoff:

$$a_i^* = \arg\max_{a_i} \mathbb{E}[U_i(a_1, a_2)]$$

This is a competitive system. The warehouse robots share a single team reward based on joint output, so they jointly optimize:

$$\max_{\pi_1,\ldots,\pi_n} \mathbb{E}\Big[\sum_t \gamma^t R(s_t, a_{1,t}, \ldots, a_{n,t})\Big]$$

This is a cooperative system.

2.12 Key Terms Glossary

Term	Meaning
Agent	A system that observes, decides, and acts to achieve a goal.
Environment	Everything around the agent that it can observe and interact with.
Percept (p)	The information the agent receives from the environment at a given instant.
Action (a)	What the agent does after making a decision.
Reward (R)	Feedback telling the agent how good or bad its action was.

Term	Meaning
Policy (f / π)	The strategy mapping situations to actions.
Utility (U)	A numeric score expressing how desirable a state or outcome is.
Discount factor (γ)	A value in $[0, 1]$ controlling how much the agent values future reward over immediate reward.
State (s)	The current situation of the environment.
Transition function/probability (P)	Rules or probabilities describing how an action changes the current state.
Belief state ($b(s_t)$)	The agent's probabilistic best guess about the true (unobservable) state.
Rationality	Choosing the action expected to give the best outcome (maximize expected utility).
Expected Utility	$\sum_{s'} P(s' \mid s, a)\, U(s')$ – the probability-weighted average utility of an action.
Multi-Agent System (MAS)	An environment in which two or more agents interact.
Cooperative agents	Agents that share a goal and maximize a joint/team reward.
Competitive agents	Agents with conflicting goals, each maximizing its own payoff.
Exploitation	Using an action already known to work well.

Term	Meaning
Exploration	Trying a new action to discover a potentially better one.

2.13 Chapter Summary

- An **intelligent agent** perceives its environment, reasons/decides, and acts to achieve a goal; the **environment** is everything the agent can observe and interact with.
- The interaction loop is: Environment → Percept → Agent → Action → Environment (extended with Reward when learning is involved). A **policy** maps situations to actions.
- An agent is formally (S, A, P, f, U): states, actions, transition function, policy, and utility function. The optimal policy is $f^* = \arg\max_f \mathbb{E}[\sum_t \gamma^t U(s_t) \mid f]$, where $\gamma \in [0, 1]$ is the discount factor.
- Perception is modelled as $p = g(s)$ (or $p_t = g(s_t) + \epsilon_t$ with sensor noise); action updates the state via $s_{t+1} = f(s_t, a_t)(+\eta_t)$.
- Five agent types, in increasing sophistication: **Reflex** ($f(p) = a$), **Model-Based** ($\hat{s}_{t+1} = h(\hat{s}_t, a_t, p_{t+1})$), **Goal-Based** ($g(s) = \text{True}$), **Utility-Based** ($f(p) = \arg\max_a \mathbb{E}[U(s_{t+1}) \mid s_t, a]$), and **Learning** ($\pi^* = \arg\max_\pi \mathbb{E}[\sum_t \gamma^t R(s_t + a_t)]$).
- An environment is formally (S, A, P, R), with transition probability $P(s' \mid s, a)$ and reward function $R(s, a, s')$. Environments are classified as **discrete** vs **continuous** (countable vs. real-valued states/actions) and **static** vs **dynamic** (does/does not change on its own).
- **Exploration** (trying new actions) trades off against **exploitation** (using known-good actions) in dynamic environments.
- Under uncertainty, an agent maintains a **belief state** $b(s_t)$, its best probabilistic guess about the true state.
- In a **Multi-Agent System**, agents may be **cooperative** (maximize a shared team reward) or **competitive** (each maximizes its own payoff), and they com-

municate via messages that update each other's belief states.

- **Rationality** means choosing the action that maximizes **Expected Utility**: $\mathbb{E}[U \mid s, a] = \sum_{s'} P(s' \mid s, a)\, U(s')$, and the rational choice is $a^* = \arg\max_a \mathbb{E}[U \mid s, a]$.